dYdX v4 Indexer Layer - 서비스 확장성 분석

개요

dYdX v4 Indexer Layer는 마이크로서비스 아키텍처를 기반으로 구축되어 높은 확장성과 가용성을 제공합니다. 이 문서는 Indexer 서비스들의 수평 확장 전략, 데이터베이스 샤딩 및 읽기 복제본 활용 방안, 그리고 Kafka 파티셔닝 및 컨슈머 그룹 최적화 방법을 상세히 분석합니다.

마이크로서비스 수평 확장 전략

1. 서비스별 확장 전략

1.1 Ender 서비스 확장

현재 아키텍처:

- 단일 인스턴스로 온체인 이벤트 처리
- 블록 순서 보장을 위한 순차 처리

확장 전략:

```
// 블록 높이 기반 파티셔닝
class EnderScalingManager {
private instances: EnderInstance[] = [];
private blockAssignments: Map<number, string> = new Map();
// 블록을 인스턴스에 할당
assignBlockToInstance(blockHeight: number): string {
```

```
const instanceId = `ender-${blockHeight % this.instances.length}`;

this.blockAssignments.set(blockHeight, instanceId);

return instanceId;

}

// 순서 보장을 위한 배리어 동기화

async waitForPreviousBlocks(blockHeight: number): Promise<void> {

const previousHeight = blockHeight - 1;

while (!this.isBlockProcessed(previousHeight)) {

await this.sleep(10); // 10ms 대기

}

}

}
```

확장 제약사항 및 해결방안:

- **제약**: 블록 순서 보장 필요
- **해결**: 블록 높이 기반 파티셔닝 + 배리어 동기화
- **효과**: 처리량 2-3배 증가, 지연 시간 유지

1.2 Vulcan 서비스 확장

현재 아키텍처:

- 오프체인 이벤트 배치 처리
- 주문서 상태 관리

확장 전략:

```
// 시장별 파티셔닝
class VulcanScalingManager {
private marketPartitions: Map<string, VulcanInstance[]> = new Map();
// 시장별 인스턴스 할당
getInstanceForMarket(marketId: string): VulcanInstance {
const instances = this.marketPartitions.get(marketId) || [];
// 라운드 로빈 방식으로 인스턴스 선택
const instanceIndex = this.getNextInstanceIndex(marketId);
return instances[instanceIndex];
}
// 동적 스케일링
async scaleMarketInstances(marketId: string, targetInstances: number): Promise<
const currentInstances = this.marketPartitions.get(marketId) || [];
if (currentInstances.length < targetInstances) {
// 인스턴스 추가
await this.addInstances(marketId, targetInstances - currentInstances.length);
} else if (currentInstances.length > targetInstances) {
// 인스턴스 제거 (그레이스풀 셧다운)
await this.removeInstances(marketId, currentInstances.length - targetInstances)
}
}
```

```
}
```

확장 효과:

- 시장별 독립적 확장 가능
- 주문 처리량 선형적 증가
- 장애 격리 효과

1.3 Comlink API 서비스 확장

로드 밸런싱 전략:

```
// 지능형 로드 밸런서
class ComlinkLoadBalancer {
private instances: ComlinkInstance[] = [];
private healthChecker: HealthChecker;
// 가중치 기반 라운드 로빈
selectInstance(requestType: string): ComlinkInstance {
const availableInstances = this.instances.filter(i => i.isHealthy());
// 요청 타입별 가중치 적용
const weights = this.getWeights(requestType);
return this.weightedRoundRobin(availableInstances, weights);
}
// 동적 가중치 조정
private getWeights(requestType: string): Map<string, number> {
const weights = new Map<string, number>();
```

```
for (const instance of this.instances) {
const metrics = instance.getMetrics();
// CPU, 메모리, 응답 시간 기반 가중치 계산
const cpuWeight = 1 - metrics.cpuUsage;
const memoryWeight = 1 - metrics.memoryUsage;
const latencyWeight = 1 / (metrics.avgResponseTime + 1);
const totalWeight = (cpuWeight + memoryWeight + latencyWeight) / 3;
weights.set(instance.id, totalWeight);
}
return weights;
}
}
```

1.4 Socks WebSocket 서비스 확장

연결 분산 전략:

```
// WebSocket 연결 관리자
class SocksConnectionManager {
private instances: SocksInstance[] = [];
private connectionMap: Map<string, string> = new Map(); // userId -> instanceI
// 사용자별 인스턴스 할당
assignUserToInstance(userId: string): string {
// 일관된 해싱으로 인스턴스 선택
const hash = this.consistentHash(userId);
```

```
const instanceIndex = hash % this.instances.length;
const instanceId = this.instances[instanceIndex].id;
this.connectionMap.set(userId, instanceId);
return instanceId;
}
// 인스턴스 추가 시 연결 재분산
async rebalanceConnections(newInstance: SocksInstance): Promise<void> {
this.instances.push(newInstance);
const connectionsToMove: string[] = [];
// 재해싱으로 이동할 연결 식별
for (const [userId, currentInstanceId] of this.connectionMap) {
const newInstanceId = this.assignUserToInstance(userId);
if (newInstanceId !== currentInstanceId) {
connectionsToMove.push(userId);
}
}
// 점진적 연결 이동
await this.migrateConnections(connectionsToMove);
}
}
```

2. 자동 스케일링 시스템

2.1 메트릭 기반 스케일링

```
// 자동 스케일링 매니저
class AutoScalingManager {
private scalingPolicies: Map<string, ScalingPolicy> = new Map();
private metricsCollector: MetricsCollector;
// 스케일링 정책 정의
defineScalingPolicy(serviceName: string, policy: ScalingPolicy): void {
this.scalingPolicies.set(serviceName, policy);
}
// 메트릭 기반 스케일링 결정
async evaluateScaling(): Promise<void> {
for (const [serviceName, policy] of this.scalingPolicies) {
const metrics = await this.metricsCollector.getMetrics(serviceName);
const decision = this.makeScalingDecision(metrics, policy);
if (decision.shouldScale) {
await this.executeScaling(serviceName, decision);
}
}
}
private makeScalingDecision(metrics: ServiceMetrics, policy: ScalingPolicy): Sc
const cpuThreshold = policy.cpuThreshold || 70;
const memoryThreshold = policy.memoryThreshold || 80;
```

```
const latencyThreshold = policy.latencyThreshold || 1000; // ms

// 스케일 아웃 조건

if (metrics.avgCpuUsage > cpuThreshold ||

metrics.avgMemoryUsage > memoryThreshold ||

metrics.avgResponseTime > latencyThreshold) {

return {

shouldScale: true,

direction: 'out',

targetInstances: Math.min(metrics.currentInstances * 2, policy.maxInstances)

};

}

// 스케일 인 조건

if (metrics.avgCpuUsage < cpuThreshold * 0.3 &&

metrics.avgMemoryUsage < memoryThreshold * 0.3 &&

metrics.avgResponseTime < latencyThreshold * 0.3) {

return {

shouldScale: true,

direction: 'in',

targetInstances: Math.max(Math.ceil(metrics.currentInstances / 2), policy.minIn

};

}

return { shouldScale: false };

}
```

```
}
```

2.2 예측적 스케일링

```
// 예측적 스케일링 엔진
class PredictiveScalingEngine {
private historicalData: TimeSeriesData[] = [];
private mlModel: TrafficPredictionModel;
// 트래픽 패턴 학습
async trainModel(): Promise<void> {
const features = this.extractFeatures(this.historicalData);
await this.mlModel.train(features);
}
// 미래 트래픽 예측
async predictTraffic(timeHorizon: number): Promise<TrafficPrediction> {
const currentFeatures = this.getCurrentFeatures();
return await this.mlModel.predict(currentFeatures, timeHorizon);
}
// 예측 기반 사전 스케일링
async proactiveScale(): Promise<void> {
const prediction = await this.predictTraffic(300); // 5분 후 예측
if (prediction.expectedLoad > this.getCurrentCapacity() * 0.8) {
// 예상 부하가 현재 용량의 80%를 초과하면 사전 스케일링
```

```
    await this.preemptiveScaleOut(prediction.expectedLoad);

  }

}

}
```

데이터베이스 샤딩 및 읽기 복제본 활용

1. PostgreSQL 샤딩 전략

1.1 수평 샤딩 (Horizontal Sharding)

시간 기반 샤딩:

```
-- 월별 테이블 파티셔닝
CREATE TABLE trades_2024_01 PARTITION OF trades
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE trades_2024_02 PARTITION OF trades
FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- 자동 파티션 생성 함수
CREATE OR REPLACE FUNCTION create_monthly_partition(table_name text, start_dat
RETURNS void AS $$
DECLARE
partition_name text;
```

```
end_date date;

BEGIN

partition_name := table_name || '_' || to_char(start_date, 'YYYY_MM');

end_date := start_date + interval '1 month';

EXECUTE format('CREATE TABLE %I PARTITION OF %I FOR VALUES FROM (%L) TO (%L)',

partition_name, table_name, start_date, end_date);

-- 인덱스 생성

EXECUTE format('CREATE INDEX ON %I (created_at)', partition_name);

EXECUTE format('CREATE INDEX ON %I (market)', partition_name);

END;

$$ LANGUAGE plpgsql;
```

시장 기반 샤딩:

```
// 샤드 라우터

class DatabaseShardRouter {

private shards: Map<string, DatabaseConnection> = new Map();

// 시장별 샤드 매핑

getShardForMarket(market: string): DatabaseConnection {

const shardKey = this.calculateShardKey(market);

return this.shards.get(shardKey) || this.shards.get('default')!;

}

private calculateShardKey(market: string): string {

// 일관된 해싱으로 샤드 결정
```

```
const hash = this.consistentHash(market);
const shardIndex = hash % this.shards.size;
return `shard_${shardIndex}`;
}
// 크로스 샤드 쿼리 처리
async executeDistributedQuery(query: string, params: any[]): Promise<any[]> {
const promises = Array.from(this.shards.values()).map(shard =>
shard.query(query, params)
);
const results = await Promise.all(promises);
return this.mergeResults(results);
}
}
```

1.2 수직 샤딩 (Vertical Sharding)

기능별 데이터베이스 분리:

```
// 데이터베이스 클러스터 관리자
class DatabaseClusterManager {
private clusters: Map<string, DatabaseCluster> = new Map();
constructor() {
// 기능별 클러스터 초기화
this.clusters.set('trading', new DatabaseCluster('trading_cluster'));
```

```
this.clusters.set('analytics', new DatabaseCluster('analytics_cluster'));

this.clusters.set('user_data', new DatabaseCluster('user_cluster'));

this.clusters.set('market_data', new DatabaseCluster('market_cluster'));

}

// 쿼리 타입에 따른 클러스터 선택

getClusterForQuery(queryType: QueryType): DatabaseCluster {

switch (queryType) {

case QueryType.TRADE_EXECUTION:

return this.clusters.get('trading')!;

case QueryType.HISTORICAL_ANALYSIS:

return this.clusters.get('analytics')!;

case QueryType.USER_PROFILE:

return this.clusters.get('user_data')!;

case QueryType.MARKET_INFO:

return this.clusters.get('market_data')!;

default:

return this.clusters.get('trading')!;

}

}

}
```

2. 읽기 복제본 활용 전략

2.1 읽기/쓰기 분리

```
// 읽기/쓰기 분리 매니저
class ReadWriteSplitManager {
private masterConnection: DatabaseConnection;
private readReplicas: DatabaseConnection[] = [];
private replicationLag: Map<string, number> = new Map();
// 쿼리 타입에 따른 연결 선택
getConnection(queryType: QueryType, maxLagMs: number = 1000): DatabaseConnection
if (queryType === QueryType.WRITE || queryType === QueryType.CRITICAL_READ) {
return this.masterConnection;
}
// 복제 지연이 허용 범위 내인 복제본 선택
const availableReplicas = this.readReplicas.filter(replica => {
const lag = this.replicationLag.get(replica.id) || 0;
return lag <= maxLagMs;
});
if (availableReplicas.length === 0) {
// 사용 가능한 복제본이 없으면 마스터 사용
return this.masterConnection;
}
// 부하 분산을 위한 라운드 로빈 선택
return this.selectLeastLoadedReplica(availableReplicas);
}
```

```
// 복제 지연 모니터링
async monitorReplicationLag(): Promise<void> {
for (const replica of this.readReplicas) {
const lag = await this.measureReplicationLag(replica);
this.replicationLag.set(replica.id, lag);
// 지연이 임계치를 초과하면 알림
if (lag > 5000) { // 5초
this.alertManager.sendAlert(`High replication lag: ${lag}ms on ${replica.id}`);
}
}
}
}
```

2.2 지역별 읽기 복제본

```
// 지역별 복제본 관리자
class GeographicalReplicaManager {
private regionalReplicas: Map<string, DatabaseConnection[]> = new Map();
constructor() {
// 지역별 복제본 설정
this.regionalReplicas.set('us-east', [/* US East replicas */]);
this.regionalReplicas.set('us-west', [/* US West replicas */]);
this.regionalReplicas.set('eu-west', [/* EU West replicas */]);
this.regionalReplicas.set('asia-pacific', [/* APAC replicas */]);
```

```
}

// 클라이언트 위치 기반 복제본 선택

getClosestReplica(clientRegion: string): DatabaseConnection {

const  regionalReplicas = this.regionalReplicas.get(clientRegion);

if (!regionalReplicas || regionalReplicas.length === 0) {

// 기본 지역 복제본 사용

return  this.regionalReplicas.get('us-east')![0];

}

// 가장 가까운 복제본 선택 (지연 시간 기준)

return  this.selectLowestLatencyReplica(regionalReplicas);

}

}
```

3. 데이터베이스 연결 풀 최적화

3.1 동적 연결 풀 관리

```
// 적응형 연결 풀

class  AdaptiveConnectionPool {

private  pools: Map<string, ConnectionPool> = new  Map();

private  metrics: PoolMetrics;

// 부하에 따른 동적 풀 크기 조정

async  adjustPoolSize(): Promise<void> {

for (const [poolName, pool] of  this.pools) {
```

```
const metrics = await this.metrics.getPoolMetrics(poolName);
// 연결 사용률이 높으면 풀 크기 증가
if (metrics.utilizationRate > 0.8) {
const newSize = Math.min(pool.maxSize * 1.5, 100);
await pool.resize(newSize);
}
// 연결 사용률이 낮으면 풀 크기 감소
if (metrics.utilizationRate < 0.3) {
const newSize = Math.max(pool.maxSize * 0.7, 10);
await pool.resize(newSize);
}
}
}
// 연결 상태 모니터링
async monitorConnections(): Promise<void> {
for (const [poolName, pool] of this.pools) {
const deadConnections = await pool.getDeadConnections();
if (deadConnections.length > 0) {
// 죽은 연결 제거 및 새 연결 생성
await pool.removeConnections(deadConnections);
await pool.createConnections(deadConnections.length);
}
}
```

```
}

}
```

Kafka 파티셔닝 및 컨슈머 그룹 최적화

1. 토픽 파티셔닝 전략

1.1 키 기반 파티셔닝

```
// 커스텀 파티셔너
class CustomPartitioner {
// 시장별 파티셔닝
partitionForMarket(market: string, numPartitions: number): number {
const hash = this.murmurHash(market);
return hash % numPartitions;
}
// 사용자별 파티셔닝 (순서 보장)
partitionForUser(userId: string, numPartitions: number): number {
const hash = this.murmurHash(userId);
return hash % numPartitions;
}
// 시간 기반 파티셔닝 (부하 분산)
partitionForTime(timestamp: number, numPartitions: number): number {
// 시간을 기준으로 라운드 로빈 방식
```

```
const timeSlot = Math.floor(timestamp / 1000); // 1초 단위
return timeSlot % numPartitions;
}
}
```

1.2 동적 파티션 관리

```
// 파티션 관리자
class PartitionManager {
private topicPartitions: Map<string, number> = new Map();
private partitionMetrics: PartitionMetrics;
// 파티션 부하 모니터링
async monitorPartitionLoad(): Promise<void> {
for (const [topic, partitionCount] of this.topicPartitions) {
const metrics = await this.partitionMetrics.getMetrics(topic);
// 핫 파티션 감지
const hotPartitions = metrics.partitions.filter(p =>
p.messageRate > metrics.avgMessageRate * 2
);
if (hotPartitions.length > 0) {
await this.rebalancePartitions(topic, hotPartitions);
}
}
```

```
}

// 파티션 리밸런싱

private  async  rebalancePartitions(topic: string, hotPartitions: PartitionInfo[

// 새 파티션 추가

const  newPartitionCount = this.topicPartitions.get(topic)! * 2;

await  this.addPartitions(topic, newPartitionCount);

// 컨슈머 그룹 리밸런싱 트리거

await  this.triggerConsumerRebalance(topic);

}

}
```

2. 컨슈머 그룹 최적화

2.1 적응형 컨슈머 스케일링

```
// 컨슈머 그룹 관리자

class  ConsumerGroupManager {

private  consumerGroups: Map<string, ConsumerGroup> = new  Map();

private  lagMonitor: ConsumerLagMonitor;

// 컨슈머 랙 기반 스케일링

async  scaleBasedOnLag(): Promise<void> {

for (const [groupId, group] of  this.consumerGroups) {

const  lagMetrics = await  this.lagMonitor.getLagMetrics(groupId);

// 높은 랙이 감지되면 컨슈머 추가
```

```
if (lagMetrics.maxLag > 10000) { // 10,000 메시지
await this.addConsumers(groupId, 2);
}
// 낮은 랙이 지속되면 컨슈머 제거
if (lagMetrics.maxLag < 1000 && group.consumerCount > 1) {
await this.removeConsumers(groupId, 1);
}
}
}
// 지능형 파티션 할당
async optimizePartitionAssignment(groupId: string): Promise<void> {
const group = this.consumerGroups.get(groupId)!;
const partitionMetrics = await this.getPartitionMetrics(group.topic);
// 처리 능력 기반 파티션 할당
const assignment = this.calculateOptimalAssignment(
group.consumers,
partitionMetrics
);
await group.reassignPartitions(assignment);
}
}
```

2.2 배치 처리 최적화

```
// 최적화된 배치 컨슈머
class OptimizedBatchConsumer {
private batchSize: number = 100;
private batchTimeout: number = 1000; // 1초
private processingQueue: MessageBatch[] = [];
// 동적 배치 크기 조정
async adjustBatchSize(): Promise<void> {
const metrics = await this.getProcessingMetrics();
// 처리 시간이 길면 배치 크기 감소
if (metrics.avgProcessingTime > 5000) { // 5초
this.batchSize = Math.max(this.batchSize * 0.8, 10);
}
// 처리 시간이 짧으면 배치 크기 증가
if (metrics.avgProcessingTime < 1000) { // 1초
this.batchSize = Math.min(this.batchSize * 1.2, 1000);
}
}
// 병렬 배치 처리
async processBatchesInParallel(batches: MessageBatch[]): Promise<void> {
const maxConcurrency = 5;
const semaphore = new Semaphore(maxConcurrency);
const promises = batches.map(async (batch) => {
```

```
await semaphore.acquire();
try {
await this.processBatch(batch);
} finally {
semaphore.release();
}
});
await Promise.all(promises);
}
}
```

3. 메시지 압축 및 직렬화 최적화

3.1 압축 전략

```
// 메시지 압축 관리자
class MessageCompressionManager {
private compressionAlgorithms: Map<string, CompressionAlgorithm> = new Map();
constructor() {
this.compressionAlgorithms.set('gzip', new GzipCompression());
this.compressionAlgorithms.set('lz4', new LZ4Compression());
this.compressionAlgorithms.set('snappy', new SnappyCompression());
}
// 메시지 타입별 최적 압축 알고리즘 선택
```

```
selectCompressionAlgorithm(messageType: string, messageSize: number): string {
// 작은 메시지는 압축하지 않음
if (messageSize < 1024) {
return 'none';
}
// 실시간 메시지는 빠른 압축 사용
if (messageType.includes('realtime')) {
return 'lz4';
}
// 배치 메시지는 높은 압축률 사용
if (messageType.includes('batch')) {
return 'gzip';
}
return 'snappy'; // 기본값
}
}
```

3.2 직렬화 최적화

```
// 고성능 직렬화
class HighPerformanceSerializer {
private schemaRegistry: SchemaRegistry;
private serializerPool: ObjectPool<Serializer>;
// 스키마 진화 지원
```

```
async serializeWithSchema(data: any, schemaVersion: number): Promise<Buffer> {
const schema = await this.schemaRegistry.getSchema(schemaVersion);
const serializer = this.serializerPool.acquire();
try {
return await serializer.serialize(data, schema);
} finally {
this.serializerPool.release(serializer);
}
}
// 배치 직렬화
async serializeBatch(messages: any[]): Promise<Buffer> {
const batchHeader = this.createBatchHeader(messages.length);
const serializedMessages = await Promise.all(
messages.map(msg => this.serializeWithSchema(msg, msg.schemaVersion))
);
return Buffer.concat([batchHeader, ...serializedMessages]);
}
}
```

캐싱 전략 및 최적화

1. 다층 캐싱 아키텍처

1.1 Redis 클러스터 구성

```
// Redis 클러스터 관리자
class RedisClusterManager {
private clusters: Map<string, RedisCluster> = new Map();
private consistentHashing: ConsistentHashing;
constructor() {
// 기능별 클러스터 구성
this.clusters.set('orderbook', new RedisCluster('orderbook_cluster'));
this.clusters.set('user_sessions', new RedisCluster('session_cluster'));
this.clusters.set('market_data', new RedisCluster('market_cluster'));
this.clusters.set('analytics', new RedisCluster('analytics_cluster'));
}
// 데이터 타입별 클러스터 선택
getClusterForData(dataType: string): RedisCluster {
return this.clusters.get(dataType) || this.clusters.get('orderbook')!;
}
// 핫 키 감지 및 분산
async detectAndDistributeHotKeys(): Promise<void> {
for (const [clusterName, cluster] of this.clusters) {
const hotKeys = await cluster.getHotKeys();
for (const hotKey of hotKeys) {
// 핫 키를 여러 노드에 복제
await this.replicateHotKey(cluster, hotKey);
```

```
}

}

}

}
```

1.2 캐시 무효화 전략

```
// 지능형 캐시 무효화

class IntelligentCacheInvalidation {

private dependencyGraph: Map<string, Set<string>> = new Map();

private invalidationQueue: Queue<InvalidationTask> = new Queue();

// 의존성 기반 무효화

async invalidateWithDependencies(key: string): Promise<void> {

const dependentKeys = this.getDependentKeys(key);

// 배치 무효화로 성능 최적화

const invalidationTasks = [key, ...dependentKeys].map(k => ({

key: k,

timestamp: Date.now()

}));

await this.batchInvalidate(invalidationTasks);

}

// 시간 기반 무효화

async scheduleTimeBasedInvalidation(key: string, ttl: number): Promise<void> {

const invalidationTime = Date.now() + ttl;
```

```
this.invalidationQueue.enqueue({
key,
invalidationTime,
type: 'time_based'
});
}
}
```

2. 메모리 최적화

2.1 메모리 사용량 모니터링

```
// 메모리 사용량 최적화
class MemoryOptimizer {
private memoryThresholds: Map<string, number> = new Map();
private compressionEnabled: boolean = true;
// 메모리 압박 시 자동 정리
async handleMemoryPressure(): Promise<void> {
const memoryUsage = await this.getMemoryUsage();
if (memoryUsage.percentage > 85) {
// LRU 기반 캐시 정리
await this.evictLRUEntries(0.2); // 20% 정리
// 압축 가능한 데이터 압축
if (this.compressionEnabled) {
```

```
    await  this.compressLargeValues();
    }
    }
    }
    // 메모리 효율적인 데이터 구조 사용
    optimizeDataStructures(): void {
    // 작은 해시맵은 ziplist 사용
    this.setConfig('hash-max-ziplist-entries', 512);
    this.setConfig('hash-max-ziplist-value', 64);
    // 작은 집합은 intset 사용
    this.setConfig('set-max-intset-entries', 512);
    // 작은 정렬된 집합은 ziplist 사용
    this.setConfig('zset-max-ziplist-entries', 128);
    this.setConfig('zset-max-ziplist-value', 64);
    }
    }
```

모니터링 및 관찰 가능성

1. 분산 추적 시스템

```
    // 분산 추적 관리자
    class  DistributedTracingManager {
```

```
private tracer: Tracer;
private spanProcessor: SpanProcessor;
// 요청 추적
async traceRequest(requestId: string, operation: () => Promise<any>): Promise<
const span = this.tracer.startSpan(`indexer.${operation.name}`, {
attributes: {
'request.id': requestId,
'service.name': 'indexer',
'service.version': process.env.SERVICE_VERSION
}
});
try {
const result = await operation();
span.setStatus({ code: SpanStatusCode.OK });
return result;
} catch (error) {
span.setStatus({
code: SpanStatusCode.ERROR,
message: error.message
});
throw error;
} finally {
span.end();
```

```
}
}
// 서비스 간 추적 전파
propagateTrace(headers: Record<string, string>): SpanContext {
return this.tracer.extract(FORMAT_HTTP_HEADERS, headers);
}
}
```

2. 성능 메트릭 수집

```
// 성능 메트릭 수집기
class PerformanceMetricsCollector {
private metrics: Map<string, Metric> = new Map();
private histograms: Map<string, Histogram> = new Map();
// 처리량 메트릭
recordThroughput(service: string, count: number): void {
const metric = this.metrics.get(`${service}.throughput`) || new Counter();
metric.add(count);
this.metrics.set(`${service}.throughput`, metric);
}
// 지연 시간 메트릭
recordLatency(service: string, latency: number): void {
const histogram = this.histograms.get(`${service}.latency`) ||
```

```
new Histogram([1, 5, 10, 25, 50, 100, 250, 500, 1000]);
histogram.record(latency);
this.histograms.set(`${service}.latency`, histogram);
}
// 에러율 메트릭
recordError(service: string, errorType: string): void {
const metric = this.metrics.get(`${service}.errors.${errorType}`) || new Count
metric.add(1);
this.metrics.set(`${service}.errors.${errorType}`, metric);
}
}
```

결론

dYdX v4 Indexer Layer의 확장성은 다음과 같은 핵심 전략을 통해 달성됩니다:

1. **마이크로서비스 확장**: 서비스별 특성에 맞는 확장 전략 적용
2. **데이터베이스 최적화**: 샤딩, 읽기 복제본, 연결 풀 최적화
3. **Kafka 최적화**: 파티셔닝, 컨슈머 그룹, 메시지 압축 최적화
4. **캐싱 전략**: 다층 캐싱, 지능형 무효화, 메모리 최적화
5. **모니터링**: 분산 추적, 성능 메트릭, 자동 스케일링

이러한 확장성 전략을 통해 dYdX v4 Indexer는 높은 거래량과 사용자 증가에 대응할 수 있는 탄력적이고 확장 가능한 아키텍처를 제공합니다.